```cpp
#pragma once
#include <iostream>
#include <vector>
#include <cmath>
#include "matrix.hpp"

using namespace std;
class capaEmbedding {
private:
    Matriz E;           // Matriz de pesos (vocab_size x dim_embedding)
    Matriz mE, vE;      // Momentos de Adam para la tabla de embeddings

    size_t vocabSize_;
    size_t dimEmbedding_;
    size_t ultimoIndice_; // Guarda el índice procesado en el forward pass

    // Hiperparámetros de Adam
    double beta1 = 0.9;
    double beta2 = 0.999;
    double epsilon = 1e-8;

public:
    capaEmbedding(size_t vocabSize, size_t dimEmbedding)
        : vocabSize_(vocabSize), dimEmbedding_(dimEmbedding), ultimoIndice_(0)
    {
        // 1. La matriz E almacena todos los vectores de características
        E = Matriz(vocabSize_, dimEmbedding_);

        // 2. Acumuladores de Adam inicializados en ceros
        mE = Matriz(vocabSize_, dimEmbedding_, 0.0);
        vE = Matriz(vocabSize_, dimEmbedding_, 0.0);

        // 3. Inicialización aleatoria (Xavier/Normal estándar para embeddings)
        double limite = std::sqrt(6.0 / (vocabSize_ + dimEmbedding_));
        E.aleatorizar(-limite, limite);
    }

    ~capaEmbedding() {}

    // Forward Pass: Entrada es la ID entera del token -> Devuelve Matriz (dimEmbedding x
1)
    Matriz forward(size_t tokenIndex) {
        if (tokenIndex >= vocabSize_) {
            throw std::out_of_range("Índice de token fuera del tamaño del vocabulario.");
        }

        ultimoIndice_ = tokenIndex;
        Matriz salida(dimEmbedding_, 1);

        // Copiamos la fila 'tokenIndex' de la tabla de embeddings a un vector columna
        for (size_t j = 0; j < dimEmbedding_; ++j) {
            salida(j, 0) = E(tokenIndex, j);
        }

        return salida;
    }

    // Backward Pass: Recibe el gradiente que viene de la siguiente capa (dimEmbedding x
1)
    void backward(const Matriz& gradienteSalida, double tasaDeAprendizaje, int t) {
        // Solo actualizamos los pesos del token utilizado en el último Forward
        for (size_t j = 0; j < dimEmbedding_; ++j) {
            double g = gradienteSalida(j, 0);

            // 1. Acumular momentos para el vector específico del token
            mE(ultimoIndice_, j) = beta1 * mE(ultimoIndice_, j) + (1.0 - beta1) * g;
```

```
            vE(ultimoIndice_, j) = beta2 * vE(ultimoIndice_, j) + (1.0 - beta2) * (g * g
);

            // 2. Corrección de sesgo
            double m_hat = mE(ultimoIndice_, j) / (1.0 - std::pow(beta1, t));
            double v_hat = vE(ultimoIndice_, j) / (1.0 - std::pow(beta2, t));

            // 3. Actualización de pesos con Adam
            E(ultimoIndice_, j) -= (tasaDeAprendizaje * m_hat) / (std::sqrt(v_hat) +
epsilon);
        }
    }

    // Getter para inspeccionar los pesos de un token
    Matriz obtenerEmbedding(size_t tokenIndex) const {
        Matriz vectorColumna(dimEmbedding_, 1);
        for (size_t j = 0; j < dimEmbedding_; ++j) {
            vectorColumna(j, 0) = E(tokenIndex, j);
        }
        return vectorColumna;
    }
    void guardarBinario(std::ofstream& out) const { E.guardarBinario(out); }
    void cargarBinario(std::ifstream& in) { E.cargarBinario(in); }
};
```